

PairRankRAG: Pairwise Chunk Ranking for Retrieval-Augmented Generation

Editors: Andy Song, Bo Han and Sarah Erfani

Abstract

Retrieval-Augmented Generation (RAG) methods often retrieve text chunks independently based on vector similarity, which may fail to capture semantic dependencies across paragraphs in long-form narrative question answering. To address this issue, we propose PairRankRAG, a pairwise chunk ranking framework that uses large language models to compare the query relevance of text segments. The resulting pairwise preferences are aggregated with the Bradley–Terry model to produce a global relevance ranking. To preserve narrative coherence, the top-ranked chunks are then reordered according to their original document positions before being used for answer generation. Experiments on NarrativeQA and GutenQA show that PairRankRAG improves evidence selection for long-form question answering. On NarrativeQA, PairRankRAG improves ROUGE-L from 0.308 to 0.321, achieving a relative gain of 4.2%. On GutenQA, PairRankRAG achieves the highest LLM-based correctness score among the compared methods, although it does not consistently outperform all baselines on lexical or embedding-based metrics. These results suggest that pairwise relevance modeling can improve context selection for long-form RAG by providing more coherent and informative evidence for downstream question answering.

Keywords: Retrieval-Augmented Generation (RAG), Long-context Question Answering, Narrative Question Answering, Pairwise Relevance Modeling, LLM-based Relevance Evaluation, Global Ranking Aggregation

1. Introduction

1.1. Background

Long-context question answering (QA) remains a fundamental yet challenging problem in natural language processing, as it requires models to integrate information distributed across one or more lengthy documents [Pang et al. \(2022\)](#). Recent advances in Transformer-based large language models (LLMs) have significantly improved the ability to process extended contexts [Qin et al. \(2024\)](#); [Wang et al. \(2024\)](#). However, these models still face inherent limitations when handling extremely long documents, primarily due to context length constraints and the tendency to underutilize information distributed across distant positions in the input.

To address these limitations, retrieval-augmented generation (RAG) has emerged as a widely adopted paradigm that decouples knowledge storage from model parameters, enabling models to dynamically access relevant external information at inference time [Lewis et al. \(2020\)](#); [Sarthi et al. \(2024\)](#). In a typical RAG pipeline, a document corpus is first pre-processed by splitting documents into fixed-length or semantically bounded passages, each of which is independently encoded into a dense vector representation using a pre-trained embedding model. At query time, the query is encoded into the same embedding space, and the top-k passages are retrieved by computing approximate nearest-neighbor search—most

commonly via cosine similarity or inner product—over the passage index. The retrieved passages are then concatenated as context and provided to a generator model to produce the final answer.

While this design is effective for factoid-style queries where answers can be localized within a single passage, it rests on a fundamental assumption: that relevance can be determined independently for each passage in isolation. This assumption becomes problematic when dealing with long-form narrative documents. Narrative texts—such as novels, historical accounts, and stories—are inherently sequential and structurally interconnected, where meaningful understanding often arises from relationships across multiple passages rather than isolated segments.

Consequently, treating passages as independent retrieval units may disrupt the underlying narrative structure. Important cross-passage dependencies can be overlooked, leading to fragmented evidence and incomplete comprehension. Moreover, conventional embedding-based retrieval ranks passages by their absolute similarity to the query, a scoring strategy that may fail to capture nuanced, context-dependent semantic relationships—particularly when the relevance of a passage can only be properly assessed in relation to other candidate passages.

1.2. Motivation

In real-world long-context QA scenarios, the relevance of a passage cannot always be determined independently. Relevance is often inherently comparative: a passage may be more informative than another precisely because it provides complementary context, resolves ambiguity, or narrows the scope of potential answers. Most conventional RAG systems rely on independently computed relevance scores for each passage, without explicitly modeling interactions among candidate passages.

We argue that pairwise comparison between passages provides a more robust signal for estimating relative relevance. By explicitly comparing two passages with respect to a given query, a model can better capture fine-grained semantic differences that are difficult to measure through independent scoring. This observation motivates the design of a pairwise relevance evaluation mechanism, where passages are ranked based on comparative judgments rather than absolute similarity scores.

Furthermore, effective reasoning over long-form narratives requires not only identifying relevant passages but also preserving their original sequential structure. Simply concatenating top-ranked passages in ranking order may disrupt temporal or causal coherence, degrading the quality of generated answers. It is therefore necessary to reconstruct context in a manner that maintains the narrative order of the document while filtering out irrelevant content.

2. Related Work

2.1. Long-form Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) pipelines typically chunk documents, retrieve top-ranked passages, and condition a language model on the retrieved contexts. While effective for localized fact-seeking, this paradigm struggles in long-form question answering, where

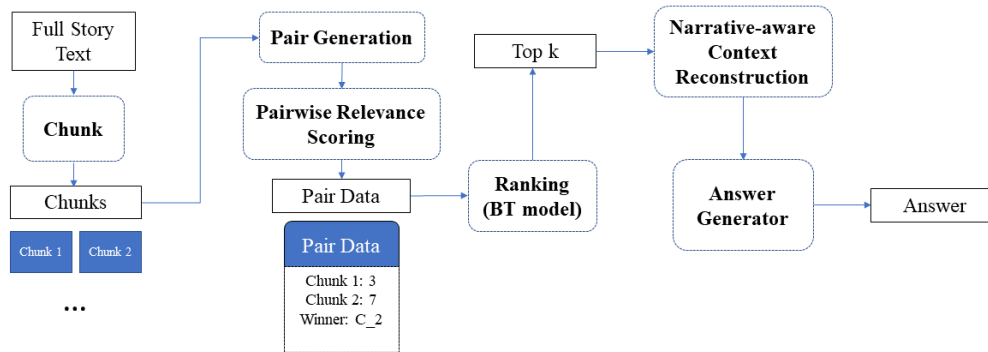


Figure 1: Pipeline of PairRankRAG. The input story is chunked and transformed into pairwise comparisons. Pairwise relevance scores are converted into preference data and ranked using the Bradley–Terry model. The top-k chunks are then reconstructed in narrative order and provided to the answer generator for final answer generation.

evidence is often distributed across multiple distant passages. As Yu et al. (2024) note, retrieving too few contexts reduces recall, while too many introduce noise that harms generation quality.

Recent work addresses this by designing retrieval structures that better preserve long-document context. RAPTOR Sarthi et al. (2024) recursively clusters and summarizes text chunks into a tree-organized index, enabling retrieval at multiple levels of abstraction for questions requiring holistic document understanding. MemWalker Chen et al. (2023) similarly constructs a hierarchical memory tree and prompts an LLM to navigate it interactively, improving explainability through the model’s traversal trajectory. ReadAgent Lee et al. (2024) takes a complementary approach inspired by human reading: it compresses documents into gist memories and retrieves original passages on demand, demonstrating that long-form QA benefits from balancing global compression with selective local access.

Graph-based methods further overcome the locality limitation of vector retrieval. GraphRAG Edge et al. (2025) builds an entity-centric knowledge graph, detects communities, and answers global queries via map-reduce summarization, showing that individually similar chunks are insufficient for queries requiring corpus-wide understanding. LightRAG Guo et al. (2024) extends this direction by integrating a dual-level retrieval paradigm over the knowledge graph, distinguishing between low-level retrieval of specific entities and high-level retrieval of broader relational themes, while supporting incremental graph updates to accommodate dynamic corpora. Both methods demonstrate that representing inter-entity dependencies through graph structures substantially improves comprehensiveness and response diversity over purely chunk-based retrieval. PairRankRAG shares this motivation but models cross-chunk relationships through pairwise relevance comparisons rather than predefined graph structures, making it well-suited for evidence selection within a single long document.

2.2. Pairwise Ranking for Cross-Chunk Relevance Modeling

Text ranking is a key component of RAG, since retrieved context quality directly affects answer generation. Pointwise rerankers score each candidate independently, requiring relevance scores to be globally calibrated—a task that is challenging for LLMs. Listwise prompting avoids this but demands a full permutation over many candidates, often producing inconsistent or incomplete outputs [Qin et al. \(2024\)](#).

Pairwise Ranking Prompting (PRP; [Qin et al. \(2024\)](#)) addresses both limitations by asking the model to compare only two passages at a time. This local preference judgment sidesteps calibration and admits flexible aggregation strategies (all-pair, sort-based, sliding-window) with different quality–cost trade-offs. PRP-Graph [Luo et al. \(2024\)](#) extends PRP by using generation probabilities to estimate preference strength and aggregating pairwise scores over a ranking graph, demonstrating that pairwise signals can be combined into a coherent global order. RankRAG [Yu et al. \(2024\)](#) further shows that ranking is not merely auxiliary: unifying context reranking with generation within a single instruction-tuned LLM substantially improves RAG performance.

PairRankRAG builds on these insights by applying pairwise ranking to long-document chunks. Rather than treating chunks as isolated candidates, it compares chunk pairs under the same query and aggregates preferences via Bradley–Terry estimation into a global relevance ranking. This design is particularly suited to long-form QA, where multiple chunks may appear partially relevant in isolation but their relative importance becomes clearer through direct comparison.

3. Method

We propose PairRankRAG, an RAG framework for long-form narrative question answering that models retrieval as a pairwise ranking task rather than scoring each fragment independently. Given a story document and a question, the framework first splits the document into chunks, compares chunk relevance using an LLM-based scorer, aggregates pairwise preferences with the Bradley–Terry model, and reconstructs a narrative-aware context for answer generation. The overall pipeline consists of five stages: document chunking, pairwise relevance scoring, ranking, narrative-aware context reconstruction, and answer generation, as illustrated in [Figure 1](#).

3.1. Document Chunking

Following prior work [Kim et al. \(2025\)](#), we first segment each full story document into fixed-length chunks based on sentence boundaries. Let a document be denoted as D , which is split into a sequence of chunks:

$$D = \{c_1, c_2, \dots, c_n\}.$$

Each chunk contains at most k tokens and preserves the original order of the story. These chunks serve as the basic retrieval units in our framework. Compared with treating the entire document as a single input, chunking allows the model to operate under the context length limitation of language models while still maintaining access to localized narrative evidence.

3.2. Pair Generation

Given a document split into n chunks, we denote the chunk set as $\mathcal{C} = \{c_1, c_2, \dots, c_n\}$. Exhaustively comparing all possible chunk pairs becomes computationally expensive for long-form documents, since the number of comparisons grows quadratically with the number of chunks. Therefore, we propose a three-stage pair construction strategy to obtain informative pairwise comparisons under a limited budget.

First, we construct a connectivity-preserving backbone graph. Let $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ be a random permutation of the chunk indices. We pair adjacent chunks in this permutation:

$$\mathcal{P}_{\text{backbone}} = \{(c_{\pi_t}, c_{\pi_{t+1}}) \mid t = 1, \dots, n-1\}.$$

This produces $n-1$ comparisons and guarantees that the comparison graph is connected. This property is important for the subsequent ranking stage, since disconnected comparison components may lead to incomparable score scales.

Second, we augment the backbone with degree-controlled random comparisons. The goal of this stage is to ensure that each chunk is compared with a small number of additional randomly selected chunks, so that the comparison graph is denser than the backbone chain while still remaining sparse.

Specifically, for each chunk c_i , we randomly sample up to d_0 other chunks as additional opponents, where d_0 controls the maximum number of extra random comparisons contributed by each chunk. Self-comparisons are not allowed, and duplicated unordered pairs are removed. Therefore, if the pair (c_i, c_j) has already been included in the backbone or has been sampled previously, it will not be added again.

Let $\mathcal{P}_{\text{random}}$ denote the set of all additional random pairs sampled in this stage. The initial comparison set is then defined as

$$\mathcal{P}_{\text{init}} = \mathcal{P}_{\text{backbone}} \cup \mathcal{P}_{\text{random}}.$$

This degree-controlled sampling prevents the comparison budget from growing quadratically. Instead of comparing every possible pair, each chunk only receives a limited number of extra random comparisons. As a result, the graph becomes more informative than a simple chain while keeping the total number of comparisons manageable.

Third, after obtaining an initial ranking, we perform a Swiss-system-inspired refinement on a candidate set selected from the top-ranked chunks [Luo et al. \(2024\)](#); [Ponomarenko et al. \(2015\)](#). In each refinement round, chunks are sorted according to their current relevance scores, and chunks with similar scores are paired for additional comparison. Specifically, we first pair adjacent chunks in the current ranking order. If an adjacent pair has already been compared, we select the nearest available unmatched opponent instead. This strategy allocates the remaining comparison budget to chunks with similar estimated relevance, which are more difficult to distinguish and more influential for the final top- k retrieval results.

Let $\mathcal{P}_{\text{swiss}}$ denote the set of additional comparison pairs generated during the Swiss-system-inspired refinement stage. The final comparison set used for pairwise relevance scoring is then defined as

$$\mathcal{P} = \mathcal{P}_{\text{init}} \cup \mathcal{P}_{\text{swiss}}.$$

Thus, $\mathcal{P}$ contains the connectivity-preserving backbone pairs, the degree-controlled random pairs, and the refinement pairs between chunks with similar estimated relevance.

3.3. Pairwise Relevance Scoring

For each question q , we construct a set of chunk pairs (c_i, c_j) from the document. Instead of independently scoring each chunk, we prompt an LLM to directly compare two chunks within the same prompt and determine which chunk is more relevant to the question. For reproducibility, the complete system prompt and user prompt are provided in Appendix A, Tables 7 and 8, respectively.

To enforce a strict preference structure, the model is required to assign different relevance scores to the two chunks, ensuring that one chunk is always ranked higher than the other. Specifically, the model outputs two relevance scores:

$$(s_i, s_j), \quad s_i, s_j \in [0, 10], \quad s_i \neq s_j$$

where s_i and s_j denote the relevance of chunks c_i and c_j to the question q , respectively.

The resulting preference observation is defined as:

$$c_i \succ c_j \quad \text{if} \quad s_i > s_j$$

and

$$c_j \succ c_i \quad \text{if} \quad s_j > s_i$$

Since ties are not allowed, every chunk pair produces a strict pairwise preference. Through this process, the LLM-generated comparisons are transformed into preference data for global ranking.

3.4. Ranking

To obtain a global ranking over all document chunks, we aggregate the pairwise preference observations using the Bradley–Terry model [Bradley and Terry \(1952\)](#). The Bradley–Terry model assumes that each chunk c_i has a latent relevance score θ_i . Given two chunks c_i and c_j , the probability that c_i is preferred over c_j is modeled as:

$$P(c_i \succ c_j) = \frac{\exp(\theta_i)}{\exp(\theta_i) + \exp(\theta_j)}.$$

The model estimates the latent relevance scores $\theta = \{\theta_1, \theta_2, \dots, \theta_n\}$ by maximizing the likelihood of the observed pairwise preferences. Chunks that consistently win pairwise comparisons receive higher latent relevance scores, while chunks that are frequently judged as less relevant receive lower scores.

After optimization, all chunks are sorted according to their estimated Bradley–Terry scores. We then select the top- k chunks as the most relevant evidence for answering the question:

$$C_{\text{top}} = \text{TopK}(\{c_1, c_2, \dots, c_n\}).$$

This ranking step transforms local pairwise relevance judgments into a global relevance ordering over the document.

3.5. Narrative-aware Context Reconstruction

Although the Bradley–Terry model provides a relevance-based ranking, directly feeding the ranked chunks into the answer generator may disrupt the original narrative structure of the story. This is particularly problematic for long-form narrative question answering, where the answer often depends not only on isolated evidence but also on the temporal and causal flow of events.

Therefore, after selecting the top- k chunks, we reconstruct the context in a narrative-aware manner. First, each selected chunk is compressed into a single sentence using an LLM. This compression step reduces irrelevant details while preserving the main information needed for answering the question. Let the compressed sentence of chunk c_i be denoted as g_i .

Then, instead of arranging the compressed chunks by their relevance scores, we reorder them according to their original positions in the document:

$$G = \{g_{i_1}, g_{i_2}, \dots, g_{i_k}\}, \quad \text{where } i_1 < i_2 < \dots < i_k.$$

This produces an ordered narrative summary that preserves the story progression while retaining the most relevant information identified by PairRank. The reconstructed context is therefore both relevance-focused and narrative-aware.

3.6. Answer Generation

Finally, the reconstructed context is provided to a downstream question answering model to generate the final answer. In our framework, we use UnifiedQA as the QA model [Khashabi et al. \(2022\)](#). The compressed narrative context and the input question are combined into a prompt and fed into the QA model, which produces the final answer.

4. Experiments

4.1. Experimental Setup

We evaluate PairRankRAG on long-form question answering tasks, where relevant evidence is often distributed across multiple distant document chunks. The goal of the experiments is to examine whether pairwise chunk ranking can improve both retrieval quality and final answer generation compared with conventional similarity-based retrieval and existing long-context RAG baselines.

For each question-document pair, the document is first split into fixed-size chunks. PairRankRAG then constructs a sparse comparison graph among chunks, uses an LLM to perform pairwise relevance scoring, aggregates the pairwise preferences with the Bradley–Terry model, and selects the top-ranked chunks as the retrieval context for the answer generator.

4.1.1. DATASET

We employ the NarrativeQA [Kočíský et al. \(2018\)](#) and GutenQA [Duarte et al. \(2024\)](#) datasets to evaluate the performance of our method on long-document question answering. NarrativeQA comprises 10,557 question-answer pairs from 391 stories, while GutenQA

NarrativeQA			
Method	ROUGE-L	CosineSim	LLM Eval
NaiveRAG	0.255	0.841	0.183
Propositionizer	0.262	0.846	0.189
RAPTOR_CT	0.298	0.854	0.241
RAPTOR_TT	0.289	0.851	0.231
LightRAG	0.240	0.841	0.182
GraphRAG	0.195	0.823	0.139
ChronoRAG	0.308	0.853	0.257
PairRankRAG	0.321	0.861	0.396

GutenQA			
Method	ROUGE-L	CosineSim	LLM Eval
NaiveRAG	0.166	0.769	0.251
Propositionizer	0.151	0.779	0.167
RAPTOR_CT	0.159	0.775	0.244
RAPTOR_TT	0.104	0.762	0.134
LightRAG	0.083	0.740	0.075
GraphRAG	0.122	0.760	0.134
ChronoRAG	0.159	0.776	0.248
PairRankRAG	0.152	0.770	0.268

Table 1: QA performance on NarrativeQA and GutenQA across ROUGE-L, CosineSim, and LLM Eval metrics. Baseline results are taken from prior work [Kim et al. \(2025\)](#). Top performance is bolded.

consists of 3,000 question-answer pairs from 100 stories. Following prior work, we use NarrativeQA under the same general evaluation setting, without constructing an additional temporal-reasoning subset. Since both datasets are constructed from lengthy narrative documents, they require models to retrieve and reason over information distributed across long contexts, making them suitable benchmarks for evaluating retrieval methods that must identify question-relevant evidence while preserving global contextual understanding.

4.1.2. EVALUATION METRICS

Following prior work on long-form RAG evaluation [Kim et al. \(2025\)](#), we report ROUGE-L [Lin \(2004\)](#), cosine similarity, and LLM-based correctness evaluation.

ROUGE-L We measure lexical answer quality using ROUGE-L, which computes the longest common subsequence overlap between the generated answer and the reference answer. We report the ROUGE-L F1 score averaged over all test examples.

Cosine Similarity Since ROUGE-L may fail to capture semantically similar answers that differ lexically, we compute cosine similarity between the generated answer and the reference answer. Specifically, both answers are encoded into dense embeddings using Snowflake model [Merrick et al. \(2024\)](#), and their cosine similarity is calculated as a semantic similarity score.

LLM-based Evaluation For semantic correctness, we use GPT-4.1 Mini Achiam et al. (2023) as the LLM judge. The judge receives the question, the generated answer, and the ground-truth answer, and outputs a binary label, *Correct* or *Wrong*. To ensure comparability with prior work, we use the same evaluation prompt as ChronoRAG Kim et al. (2025).

4.1.3. BASELINES

Following prior work on long-form RAG evaluation Kim et al. (2025), we compare PairRankRAG with representative retrieval and RAG baselines, including NaiveRAG, Propositionizer, RAPTOR_CT, RAPTOR_TT, LightRAG, GraphRAG, and ChronoRAG. These methods cover standard chunk-level retrieval, proposition-level retrieval, hierarchical retrieval, graph-based retrieval, and chronological passage assembly. The baseline results reported in Table 1 are taken from prior work Kim et al. (2025), while PairRankRAG denotes our method evaluated using the same datasets and evaluation metrics.

NaiveRAG is a standard RAG pipeline that performs chunk-level retrieval without additional document structuring Lewis et al. (2020).

RAPTOR clusters semantically similar chunks and recursively summarizes them into a tree structure for hierarchical retrieval Sarthi et al. (2024).

LightRAG constructs a lightweight entity-relation graph for efficient retrieval, using dual-level extraction and incremental updates Guo et al. (2024).

GraphRAG builds a richer graph structure with relation weighting and neighborhood assembly to support multi-hop retrieval over both high-level summaries and underlying chunks Edge et al. (2025).

Propositionizer decomposes source documents into fine-grained propositions and uses each proposition as a retrieval unit before answer generation Chen et al. (2024).

ChronoRAG organizes retrieved passages according to their chronological order to preserve temporal coherence and support time-aware answer generation Kim et al. (2025).

4.1.4. IMPLEMENTATION DETAILS

For a fair comparison with prior work Kim et al. (2025), PairRankRAG is evaluated under the same dataset, metrics, answer generator, and context budget whenever applicable. We use meta-llama-3-8B-Instruct Grattafiori et al. (2024) for pairwise relevance estimation and unifiedqa-v2-t5-3b-1363200 Khashabi et al. (2022) for answer generation. Following prior work, the retrieved context is limited to 1,500 tokens; when the ranked chunks exceed this budget, we retain the highest-ranked chunks until the limit is reached. For evaluation, we use Snowflake/snowflake-arctic-embed-l Merrick et al. (2024) to compute cosine similarity between generated and ground-truth answers. For pair generation, we set the maximum number of additional random opponents per chunk to $d_0 = 6$. After the initial Bradley–Terry ranking, we select at most 40 top-ranked chunks as candidates and perform $R = 6$ Swiss-system-inspired refinement rounds, where each round pairs chunks with similar current relevance scores. The Bradley–Terry model is then refitted on the refined candidate comparisons to obtain the final top- k chunks.

Question: Who does Maskull accept an invitation from?		
Method	Answer	Retrieved evidence
PairRank	Krag	Krag invites Maskull to join him on a journey from the Starkness Observatory. After Krag touches him, Maskull experiences a sudden physical and mental shock, which supports Krag as the correct answer.
RAPTOR	a total stranger	The retrieved text contains dialogue around Maskull’s introduction, but does not include the key evidence connecting the invitation event to Krag.
LightRAG	Mrs. Trent	The retrieved graph/context emphasizes related entities such as Mrs. Trent, Maskull, and Nightspore, but retrieves a different invitation relation.

Table 2: Case study of retrieved evidence from different methods. PairRank retrieves evidence that directly supports the correct answer, while the baselines retrieve related but insufficient context.

4.2. Main Results

Table 1 shows the main results on NarrativeQA and GutenQA. On NarrativeQA, PairRankRAG achieves the best performance across all evaluation metrics, obtaining 0.321 ROUGE-L, 0.861 cosine similarity, and 0.396 LLM Eval. Compared with the strongest baseline, ChronoRAG, PairRankRAG improves ROUGE-L from 0.308 to 0.321, cosine similarity from 0.853 to 0.861, and LLM Eval from 0.257 to 0.396. The particularly large gain in LLM-based evaluation suggests that the proposed pairwise chunk ranking strategy is effective in retrieving evidence that better supports semantically correct answers for long-form narrative question answering. On GutenQA, PairRankRAG achieves the highest LLM Eval score of 0.268, outperforming ChronoRAG and NaiveRAG, which obtain 0.248 and 0.251, respectively.

However, it does not consistently outperform other methods in terms of ROUGE-L or cosine similarity. This result should be interpreted in light of the characteristics of GutenQA and the generation behavior of UnifiedQA. Since GutenQA contains relatively longer reference answers, while UnifiedQA tends to produce concise responses [Khashabi et al. \(2022\)](#), lexical-overlap and embedding-based metrics may underestimate the quality of generated answers when they are semantically correct but shorter than the references. Overall, PairRankRAG achieves the best performance across all metrics on NarrativeQA and the highest LLM-based evaluation score on GutenQA. However, on GutenQA, its ROUGE-L and cosine similarity scores are slightly lower than the best-performing baselines. These results indicate that modeling chunk relevance through pairwise comparisons improves evidence selection, particularly in long-form QA settings where answer correctness and semantic relevance are more important than surface-level similarity.

4.3. Analysis

Effect of Pairwise Comparison To evaluate the effect of pairwise comparison, we replace the pairwise relevance scoring module with independent chunk scoring. Each chunk is scored individually with respect to the question.

As shown in Table 3, removing pairwise comparison leads to a substantial performance drop, with ROUGE-L decreasing from 0.3213 to 0.2563. This result indicates that independently assigning relevance scores to individual chunks is less effective for identifying the most useful evidence in long-form retrieval. One possible reason is that independent scoring lacks a direct comparative context, making it difficult for the model to distinguish between chunks with similar semantic relevance. In contrast, pairwise comparison forces the model to evaluate two candidate chunks relative to the same question, which provides a more discriminative signal for ranking.

Method	ROUGE-L
Full Model	0.3213
Independent Scoring	0.2563

Table 3: Analysis of the effect of pairwise relevance scoring in PairRankRAG on the NarrativeQA dataset, measured by ROUGE-L.

Effect of Bradley–Terry Ranking To examine the effect of the Bradley–Terry ranking module, we replace it with a simple average-score aggregation strategy. Specifically, instead of estimating a global ranking from pairwise preferences, each chunk is ranked according to its average relevance score across all comparisons. As shown in Table 4, removing Bradley–Terry ranking substantially decreases the ROUGE-L score from 0.3213 to 0.1800. This result indicates that directly averaging pairwise scores is insufficient for producing a reliable global ranking, since it does not explicitly model relative preferences among chunks. In contrast, Bradley–Terry ranking better aggregates pairwise comparison outcomes into a consistent global relevance order, which is crucial for selecting high-quality evidence chunks for answer generation.

Method	ROUGE-L
Full Model	0.3213
Avg. Scoring	0.1800

Table 4: Analysis of the effect of Bradley–Terry ranking in PairRankRAG on the NarrativeQA dataset, measured by ROUGE-L.

Effect of Narrative-aware Ordering To examine the effect of narrative-aware ordering, we remove the context reconstruction strategy and directly arrange the selected chunks according to their ranking scores. As shown in Table 5, the performance decreases

from 0.3213 to 0.3177 after removing narrative-aware ordering. Although the drop is relatively small, the result suggests that preserving the original narrative order still provides additional benefit for long-form question answering.

This improvement can be attributed to the nature of narrative documents, where events, character actions, and causal relations are often presented in a temporal sequence. Ranking scores are effective for identifying relevant chunks, but arranging them purely by relevance may disrupt the original story flow. In contrast, narrative-aware ordering keeps the retrieved evidence in the order in which it appears in the document, allowing the generation model to better follow the progression of events and integrate related evidence more coherently.

Method	ROUGE-L
Full Model	0.3213
w/o Narrative-aware Ordering	0.3177

Table 5: Analysis of the effect of narrative-aware ordering in PairRankRAG on the NarrativeQA dataset, measured by ROUGE-L.

Impact of Scorer Model Scale and Feasibility Boundary Analysis. To explore the boundary conditions of PairRankRAG, we replaced the `llama-3-8B-Instruct` scoring model with `gemma-3-1b-it` [Gemma Team et al. \(2025\)](#), as shown in Table 6. This configuration causes a severe performance drop, with the ROUGE-L score declining to 0.1520. We qualitatively attribute this steep decline to the increased difficulty faced by smaller models during multi-context comparative reasoning. Unlike pointwise scoring, pairwise ranking requires simultaneous attention allocation across the query and two distinct paragraphs under strict inequality constraints ($s_i \neq s_j$). This complex alignment may exceed the 1B model’s instruction-following capacity, leading to noisy preference judgments that disrupt the Bradley-Terry aggregation. Crucially, this empirical limitation further motivates the use of our proposed backbone graph and Swiss-system optimization. Since high-fidelity pairwise ranking may require larger, more expensive LLMs, minimizing the comparison budget toward near-linear complexity is important for practical and cost-effective deployment.

Model	ROUGE-L
llama-3-8B-Instruct	0.3213
gemma-3-1b-it	0.1520

Table 6: Analysis of the Effects of Different Models on ROUGE-L Scores in PairRankRAG on the NarrativeQA Dataset

4.3.1. CASE STUDY

This case illustrates two different failure modes of existing retrieval methods. RAPTOR retrieves passages that are broadly related to Maskull, but most of them describe unrelated conversations or events. As a result, the retrieved context contains many mentions of Maskull but does not clearly identify the invitation event required by the question. LightRAG, on the other hand, retrieves an explicit invitation relation between Maskull and

Mrs. Trent. However, this relation corresponds to a different event and therefore leads to an incorrect answer. These results show that retrieving passages or graph relations that are superficially related to the query is not sufficient for long-form question answering.

As shown in Table 2, PairRankRAG ranks chunks according to their relative usefulness for answering the specific question, rather than relying only on lexical overlap, entity matching, or broad semantic relatedness. This helps PairRankRAG distinguish between superficially relevant evidence and evidence that actually supports the intended answer. In this example, PairRankRAG retrieves context that better matches the target invitation event, allowing the model to identify Krag as the correct answer more reliably.

5. Conclusion

In this paper, we introduced PairRankRAG, a retrieval refinement framework for long-form question answering. Instead of relying on independent relevance scores, PairRankRAG compares document chunks pairwise and aggregates the results with the Bradley–Terry model to obtain a global evidence ranking. Experiments on NarrativeQA demonstrated that this ranking strategy improves answer generation performance compared with long-form RAG baselines, while results on GutenQA further showed gains in LLM-based correctness. Our analysis further showed that pairwise ranking helps retrieve chunks containing important narrative clues, particularly when the answer depends on sparse or distributed evidence. Overall, these results indicate that modeling relative relevance between chunks is an effective way to improve evidence selection in long-form RAG.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. doi: 10.48550/arXiv.2303.08774.
- Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. ISSN 00063444, 14643510. URL <http://www.jstor.org/stable/2334029>.
- Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. Walking down the memory maze: Beyond context limit through interactive reading. 2023.
- Tong Chen, Hongwei Wang, Sihao Chen, Wenhao Yu, Kaixin Ma, Xinran Zhao, Hongming Zhang, and Dong Yu. Dense X Retrieval: What Retrieval Granularity Should We Use? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 15159–15177, 2024.
- André V. Duarte, João DS Marques, Miguel Graça, Miguel Freire, Le Li, and André L. Oliveira. Lumberchunker: Long-form narrative document segmentation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 6473–6486, Miami, Florida, USA, 2024. Association for Computational Linguistics.

- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graphrag approach to query-focused summarization. 2025.
- Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Rame, Morgane Riviere, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. Lightrag: Simple and fast retrieval-augmented generation. 2024.
- Daniel Khashabi, Yeganeh Kordi, and Hannaneh Hajishirzi. Unifiedqa-v2: Stronger generalization via broader cross-format training. *arXiv preprint arXiv:2202.12359*, 2022.
- Byeongjeong Kim, Jeonghyun Park, Joonho Yang, and Hwanhee Lee. Chronological passage assembling in rag framework for temporal question answering, 2025. URL <https://arxiv.org/abs/2508.18748>.
- Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 6:317–328, 2018.
- Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A human-inspired reading agent with gist memory of very long contexts. In *ICML*, 2024.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81. Association for Computational Linguistics, 2004.
- Jian Luo, Xuanang Chen, Ben He, and Le Sun. Prp-graph: Pairwise ranking prompting to llms with graph aggregation for effective text re-ranking. In *ACL*, 2024.
- Luke Merrick, Danmei Xu, Gaurav Nuti, and Daniel Campos. Arctic-embed: Scalable, efficient, and accurate text embedding models. *arXiv preprint arXiv:2405.05374*, 2024. doi: 10.48550/arXiv.2405.05374.
- Richard Yuanzhe Pang, Alicia Parrish, Nitish Joshi, Nikita Nangia, Jason Phang, Angelica Chen, Vishakh Padmakumar, Johnny Ma, Jana Thompson, He He, and Samuel Bowman. QuALITY: Question answering with long input texts, yes! In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5336–5358, 2022.

Nikolay Ponomarenko, Lina Jin, Oleg Ieremeiev, Vladimir Lukin, Karen Egiazarian, Jaakko Astola, Benoit Vozel, Kacem Chehdi, Marco Carli, Federica Battisti, and C.-C. Jay Kuo. Image database tid2013: Peculiarities, results and perspectives. *Signal Processing: Image Communication*, 30:57–77, 2015. doi: 10.1016/j.image.2014.10.009.

Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. Large language models are effective text rankers with pairwise ranking prompting. 2024.

Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. Raptor: Recursive abstractive processing for tree-organized retrieval. In *ICLR*, 2024.

Chonghua Wang, Haodong Duan, Songyang Zhang, Dahua Lin, and Kai Chen. Ada-level: Evaluating long-context llms with length-adaptable benchmarks. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3712–3724, 2024.

Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. Rankrag: Unifying context ranking with retrieval-augmented generation in llms. 2024.

Appendix A. Prompt Template for Pairwise Relevance Scoring

System Prompt

You are a relevance scoring system. You will be given two paragraphs from a novel or script, along with a question.

Paragraphs are marked as:

[paragraph 1]...[/paragraph 1]

[paragraph 2]...[/paragraph 2]

The question is marked as:

[question]...[/question]

Your task is purely analytical. Only compute a numeric relevance score for each paragraph based on the question, from 0 to 10.

Score guidelines:

0–2: Completely irrelevant, no useful information for answering the question.

3–4: Weakly related background or context, but does not help answer the question.

5–6: Partially relevant, mentions related events or characters but lacks key answer information.

7–8: Highly relevant, contains important information needed to answer the question.

9–10: Directly answers the question or provides explicit answer evidence.

Important:

– If one paragraph is more relevant, its score **MUST** be higher.

– Do NOT summarize, interpret, or answer the question.

For your answer, label the score for **paragraph_1** and **paragraph_2**. Your answer must strictly follow the format below, with no additional content:

Example 1: { "paragraph_1": 3, "paragraph_2": 8 }

Example 2: { "paragraph_1": 5, "paragraph_2": 2 }

Table 7: System prompt used for pairwise relevance scoring.

User Prompt

[paragraph 1]

{chunk1}

[/paragraph 1]

[paragraph 2]

{chunk2}

[/paragraph 2]

[question]

{question}

[/question]

Score each paragraph (0–10) by usefulness for answering the question.

Ensure the more relevant paragraph has the higher score.

Your task is purely analytical. Only compute a numeric relevance score for each paragraph based on the question, from 0 to 10.

For your answer, label the score for **"paragraph_1"** and **"paragraph_2"**. Your answer must strictly follow the format below, with no additional content:

Example 1: { "paragraph_1": 8, "paragraph_2": 4 }

Example 2: { "paragraph_1": 3, "paragraph_2": 7 }

Table 8: User prompt used for pairwise relevance scoring.